📘 SQL CLASS NOTE – MODULE 4: `ORDER BY`

1. Introduction

The **`ORDER BY` clause** is used to **sort the results** of a query.

- By default, SQL doesn't guarantee order — it just returns rows in whatever sequence it finds them.
- With `ORDER BY`, you decide how the results should be arranged:
  - Alphabetically (A–Z / Z–A)
  - Numerically (smallest → largest / largest → smallest)
  - By date (oldest → newest / newest → oldest)

2. Syntax

```
SELECT column1, column2, ...
FROM table_name
ORDER BY column1 [ASC|DESC], column2 [ASC|DESC];
```

- `ASC` = Ascending order (default).
- `DESC` = Descending order.
- You can sort by **one column** or **multiple columns**.

3. Practical Examples with `SchoolDB`

Example 1: Sort students by first name (alphabetical order)

```
SELECT FirstName, LastName
FROM Students
ORDER BY FirstName ASC;
```

Example 2: Sort students by age (youngest → oldest)

```
SELECT FirstName, Age
FROM Students
ORDER BY Age ASC;
```

Example 3: Sort students by age (oldest → youngest)

```
SELECT FirstName, Age
FROM Students
ORDER BY Age DESC;
```

Example 4: Sort students by grade level, then by age

```
SELECT FirstName, LastName, GradeLevel, Age
FROM Students
ORDER BY GradeLevel ASC, Age DESC;
```

📌 First arranges by `GradeLevel`, then sorts students of the same grade by age.

Example 5: Sort courses by teacher name (A–Z)

```
SELECT CourseName, Teacher
FROM Courses
ORDER BY Teacher ASC;
```

Example 6: Sort enrollments by most recent first

```
SELECT StudentID, CourseID, EnrollmentDate
FROM Enrollments
ORDER BY EnrollmentDate DESC;
```

4. Why Use `ORDER BY` ?

- Makes reports easier to read.
- Helps group data logically (e.g., names in alphabetical order).
- Essential for preparing sorted results in dashboards and analytics.

5. Real-Life Use Cases

- **Schools**: List students alphabetically in attendance sheets.
- **Banks**: Show recent transactions (newest first).
- **Hospitals**: Display patient check-ins by date.
- **E-commerce**: Sort products by price (low → high or high → low).

6. Classwork (Practice) ✅

👉 Try these queries on **SchoolDB**:

1. Display all students sorted by **LastName** alphabetically.
2. Show students ordered by **Age**, youngest first.
3. Show all female students, ordered by **Age descending**.
4. List all courses, sorted by **CourseCode**.
5. Display students sorted by **GradeLevel**, then by **FirstName**.
6. Show enrollments sorted by **EnrollmentDate ascending** (earliest first).
7. Show students sorted by **Age descending**, but if two students have the same age, order them by **LastName ascending**.

7. Challenge Question 💡

👉 Write a query that shows the **FullName**, `Age` , and `GradeLevel` of all students:

- Sorted by **GradeLevel ascending**
- Within each grade, sorted by **Age descending**

Perfect 👍 Let's solve the **Module 4 Classwork + Challenge** step by step with real SQL queries.

✅ Solutions to Module 4: `ORDER BY`

1. Display all students sorted by **LastName** alphabetically.

```sql
SELECT FirstName, LastName, Age, GradeLevel
FROM Students
ORDER BY LastName ASC;
```

2. Show students ordered by **Age**, youngest first.

```sql
SELECT FirstName, LastName, Age
FROM Students
ORDER BY Age ASC;
```

3. Show all female students, ordered by **Age descending**.

```sql
SELECT FirstName, LastName, Age, Gender
FROM Students
WHERE Gender = 'Female'
ORDER BY Age DESC;
```

4. List all courses, sorted by **CourseCode**.

```sql
SELECT CourseName, CourseCode, Teacher
FROM Courses
ORDER BY CourseCode ASC;
```

5. Display students sorted by **GradeLevel**, then by **FirstName**.

```
SELECT FirstName, LastName, GradeLevel
FROM Students
ORDER BY GradeLevel ASC, FirstName ASC;
```

6. Show enrollments sorted by **EnrollmentDate ascending** (earliest first).

```
SELECT StudentID, CourseID, EnrollmentDate
FROM Enrollments
ORDER BY EnrollmentDate ASC;
```

7. Show students sorted by **Age descending**, but if two students have the same age, order them by **LastName ascending**.

```
SELECT FirstName, LastName, Age
FROM Students
ORDER BY Age DESC, LastName ASC;
```

💡 Challenge Question

👉 Show the **FullName**, `Age` , and `GradeLevel` of all students:

- Sorted by **GradeLevel ascending**
- Within each grade, sorted by **Age descending**

```sql
SELECT CONCAT(FirstName, ' ', LastName) AS FullName,
       Age,
       GradeLevel
FROM Students
ORDER BY GradeLevel ASC, Age DESC;
```

📌 Result:

- Students grouped by grade.
- Within each grade, older students appear first.

✅ That wraps up **Module 4: ORDER BY**.

👉 Should I move on to **Module 5: DISTINCT** with the same expanded explanation, examples, classwork, and challenge?